TaxPal

Description

TaxPal is a comprehensive personal finance and tax management platform designed specifically for freelancers, gig workers, and independent contractors. Freelancers often face Irregular income, multiple expense streams, and complex tax obligations that are difficult to track manually. TaxPal addresses these challenges by providing an all-in-one solution that helps users manage their finances efficiently, plan budgets, estimate taxes, and generate downloadable reports.

The platform allows users to record every financial transaction—both income and expenses— into a centralized system. Transactions can be manually entered or imported from bank statements. Each transaction can be categorized automatically or manually to facilitate budget tracking and tax calculation. Users can also define monthly or quarterly budgets for different categories, such as rent, groceries, or software subscriptions.

Purpose

The primary purpose of **TaxPal** is to streamline the entire **tax management lifecycle** through a secure, efficient, and user-friendly digital platform. It aims to simplify the process of tax filing, compliance, and financial record management by automating repetitive tasks, reducing manual errors, and providing real-time insights into user data.

TaxPal focuses on ensuring data security and privacy through advanced encryption standards and secure authentication mechanisms, thereby safeguarding sensitive financial information from unauthorized access. The platform is designed to enhance operational efficiency by minimizing time and effort required for tax preparation, submission, and tracking.

In addition to efficiency, TaxPal seeks to empower users—including individuals, tax professionals, and corporate finance departments—by offering intuitive tools, guided workflows, and intelligent recommendations that make tax filing straightforward and stress-free. It emphasizes accountability, allowing users to clearly understand their tax obligations, deductions, and filing status.
By addressing the inefficiencies and complexities of traditional tax filing systems, TaxPal aspire become a comprehensive and reliable digital tax management solution that promotes accuracy, trust, and convenience for all its users.

Getting Started

Prerequisites

Before setting up and running the application, ensure the following software and tools are installed on your system:

Node.js and npm
- Download and install from Node.js official website.
- npm comes bundled with Node.js.

MongoDB
- Install and set up MongoDB locally or use a cloud-hosted solution like MongoDB Atlas.

Nodemailer
- Configured Nodemailer in Node.js backend to send OTPs and notification emails.

IDE
- Use a development environment like Visual Studio Code for code editing and debugging.

Installation

Follow these steps to set up the project:

Clone the Repository

git clone -> https://github.com/springboardmentor0316/Taxpal_team4.git
cd Taxpal_team4

Install Dependencies

npm install

Running the Application

Start the Backend

Navigate to the backend directory:

cd backend

Run the backend server (ensure the backend runs on a specific port, e.g., 5000):

nodemon server.js

Start the Frontend

Navigate to the frontend directory:

cd frontend

Run the development server:

npm start

Accessing the Application

Frontend: Visit http://localhost:3000 (or the port specified during the setup).
Backend API: Accessible at http://localhost:5000 for testing endpoints.

Environment Setup for Right Resource Fit

To run the project seamlessly, you need to set up environment variables for both the backend and frontend. Create a .env file in the root directory of the backend project and add the required configurations as shown below.

Required Environment Variables

Variable	Description	Example /Default value
MONGO_URI	Connection string for MongoDB database.	mongodb+srv://Susmitha:Tax%401234@cluster0.h4wpmjr.mongodb.net/taxpal?retryWrites=true&w=majority&appname=Cluster0
JWT_SECRET	Secret key for signing JWT tokens.	supersecret123
PORT	Port number for the back end server.	5000
CLIENT_URL	URL for front end connection (CORS origin)	http://localhost:3000
EMAIL_USER	Email address for sending mails.	susmithak212004@gmail.com
EMAIL_PASS	Password or app password for the email account.	igwayxzyhfcdpjac
NODE_ENV	Application environment.	Development

Steps to Set Up the Environment

- Create a .env file in the root of your backend directory.
- Copy the example configuration above and paste it into the .env file.
- Replace placeholder values (like your-email@example.com and your-email-password) with your actual configuration
- Save and restart your backend server

Nodemailer

SMTP_HOST=smtp.gmail.com

PORT=5000

SMTP_USER=susmithak212004@gmail.com

SMTP_PASS= igwayxzyhfcdpjac

Additional Notes

- For cloud-hosted MongoDB (e.g., MongoDB Atlas), replace mongodb://localhost:27017/<db> with your cluster's connection string.
- If you're using a third-party email service, adjust the SMTP_HOST, SMTP_PORT, SMTP_USER, and SMTP_PASS accordingly.
- Keep the .env file secure and do not commit it to version control. Add .env to your .gitignore file.

Folder Structure

A well-organized folder structure helps maintain clarity and efficiency in development. Below is a suggested folder structure for your TaxPal project, leveraging the MERN stack.

```
INFOSYSPROJECT/
├── taxpal-backend/
│   ├── src/
│   │   ├── config/ # Config files (db.js)
│   │   ├── controllers/ # Handles Business logic
│   │   ├── middleware/ # Handle request
│   │   ├── models/ # MongoDB schemas
│   │   ├── routes/ # API endpoints
│   │   └── server.js # Backend entry point
│   │   ├── app.js/ # App Configuration
│   ├── package.json
│   └── .env
├── taxpal-frontend/
│   ├── public/ # Static files (HTML, assets)
│   ├── src/
│   │   ├── components/ # Reusable React components
│   │   ├── pages/ # Page-level components
│   │    ├── styles/ # Contains layout, colors, and responsive
│   │   ├── App.js # Main React component
│   │   ├── api.js # Sets up the base Axios instance
│   │   └── index.js # Frontend entry point
│   ├── package.json
├── .gitignore
├── README.md
└── LICENSE
```

Key Features

• **Income & Expense Logging:** Users can record all income sources and expenses in real time with category-based tracking. This ensures accurate financial records, helps identify spending patterns, and keeps all transactions organized for easy analysis.

• **Budget Management:** Users can set spending limits for each category and monitor their progress visually. The system provides alerts when users approach or exceed budget limits, promoting disciplined spending and better financial control.

• **Automated Tax Estimation:** The system automatically calculates quarterly taxes based on total income and applicable regional rules. It updates in real time as new income or expenses are logged, helping users stay compliant and plan their taxes efficiently.

• **Financial Reports:** Users can generate detailed income, expense, and budget summaries in downloadable PDF CSV formats. These reports are well-structured for easy understanding, simplifying tax filing and supporting effective financial planning.

• **Visual Dashboard:** The dashboard provides a clear graphical representation of the user's financial health charts and key metrics. It enables users to easily analyze income, expenses, and savings trends, making complex data simple and visually insightful.

Project Components

Here's a list of key components in the TaxPal project, along with brief descriptions of their functionality:

Login Component

Description: Enables users to securely log in and access the Taxpal platform.

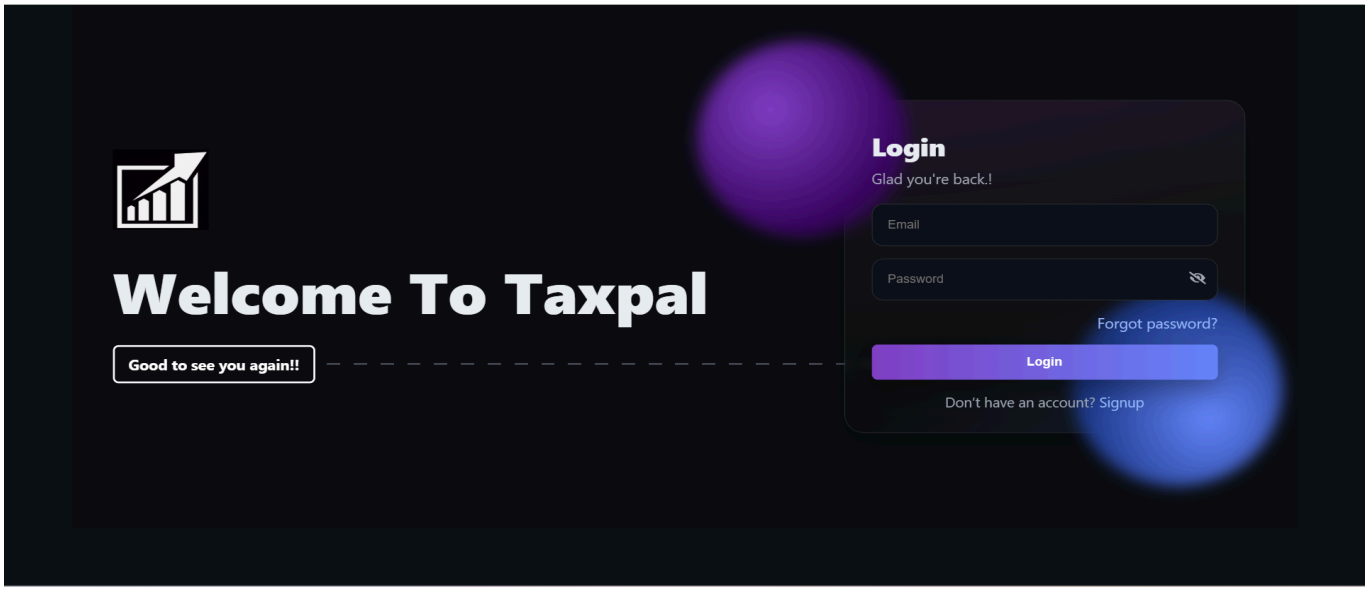
Key Features:

Secure Login: Allows users to sign in using their username and password with encrypted authentication.

Forgot Password: Provides an option to recover or reset forgotten passwords easily.

Signup Option: New users can quickly create an account to start using Taxpal services.

Welcome Interface: Displays a personalized greeting message — "Glad to see you again!" for returning users.



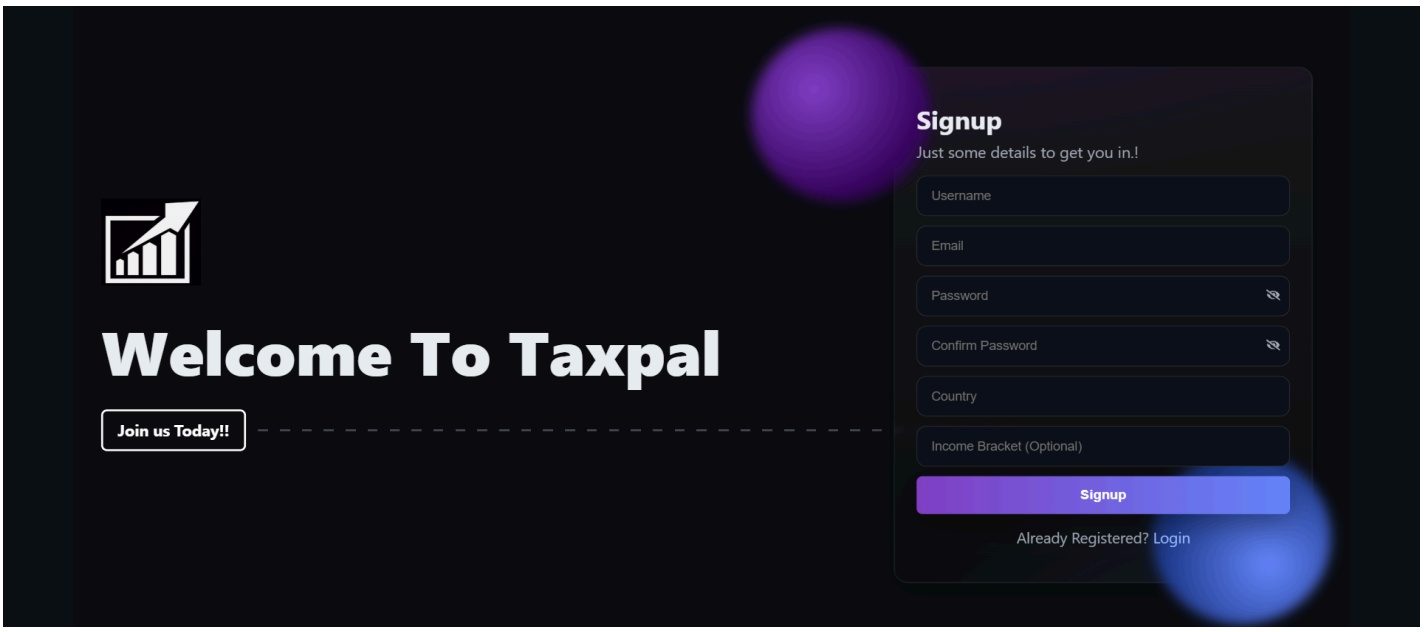
Signup Component

Enables new users to create an account and join the Taxpal platform.

Key Features:

Account Creation: Allows users to register by entering details such as username, email, and password.

Country Selection: Collects user's country information for personalized tax-related services. Income Bracket (Optional): Users can optionally provide their income range for better tax insights. Password Confirmation: Ensures account security by verifying password match before registration. Redirect to Login: Provides an option for already registered users to log in directly.



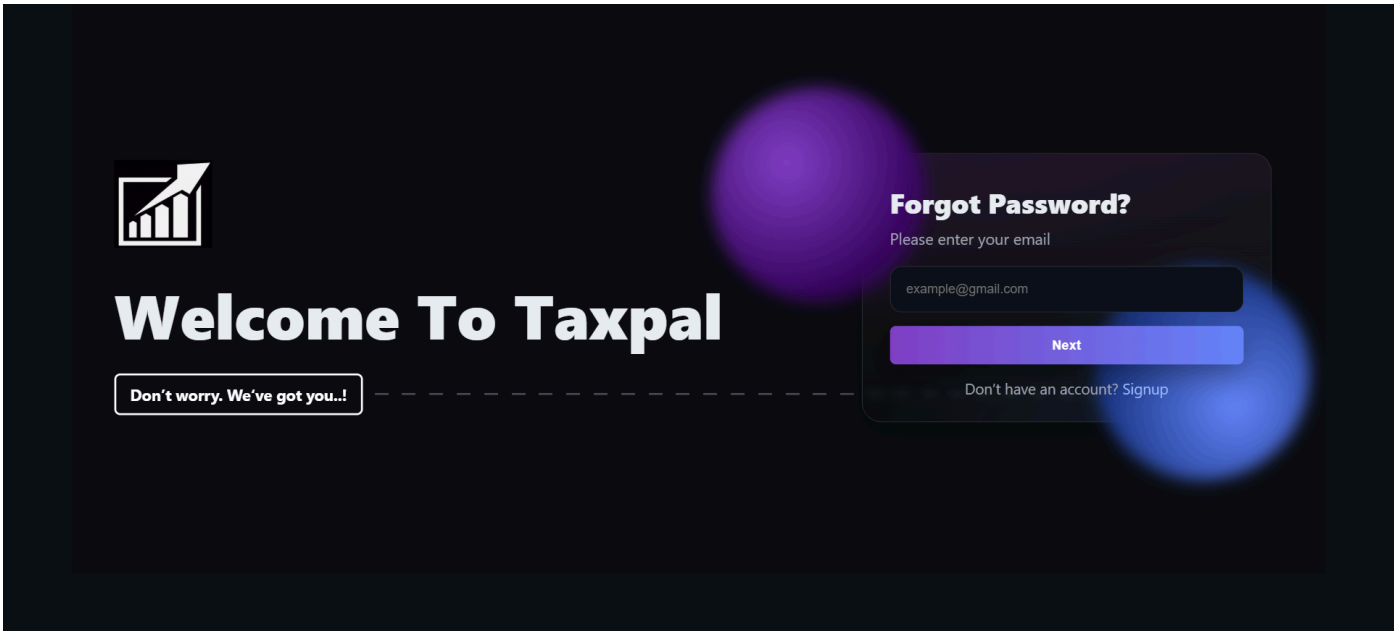
Forgot Password Component

Description: This component provides a secure and user-friendly workflow for users who cannot remember their login credentials, allowing them to regain access to their accounts.

Key Features:

Identity Verification (Email Input): The first step in the secure process. Users must enter the email address associated with their account. A confirmation message ("Please enter your email") and a placeholder ("example@mail.com") guide the user for a correct entry.

Secure Token Generation: Upon submitting the email, the system generates a unique time.

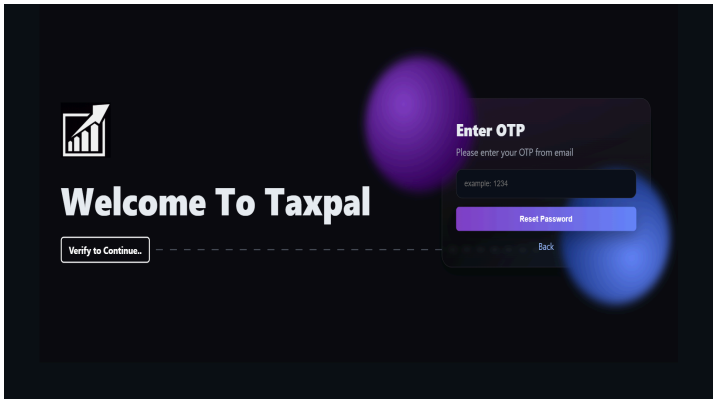


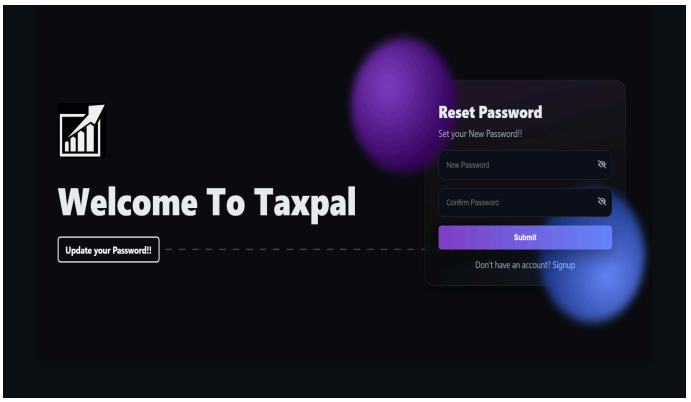
Reset Password & OTP Component

Description: Implements the functionality for password recovery using an OTP sent via email.

Key Features:

OTP Generation: Generates a one-time password (OTP) and sends it to the user's email via Gmail.
OTP Validation: Users input the OTP received in their email to verify their identity.
Password Reset: Allows users to set a new password after OTP verification.
Security: Ensures the OTP is valid for a limited time and can only be used on an OTP page.





Dashboard Component

This component provides users with a comprehensive overview of their financial activities, offering both high-level insights and detailed transaction management in a single interface.

Key Features:

Multi-module Navigation: Sidebar provides quick access to key financial sections including Budgets, Tax Estimator, Reports, and Settings for complete financial management.

Transaction Overview: Displays recent financial transactions in a structured table format with key details:

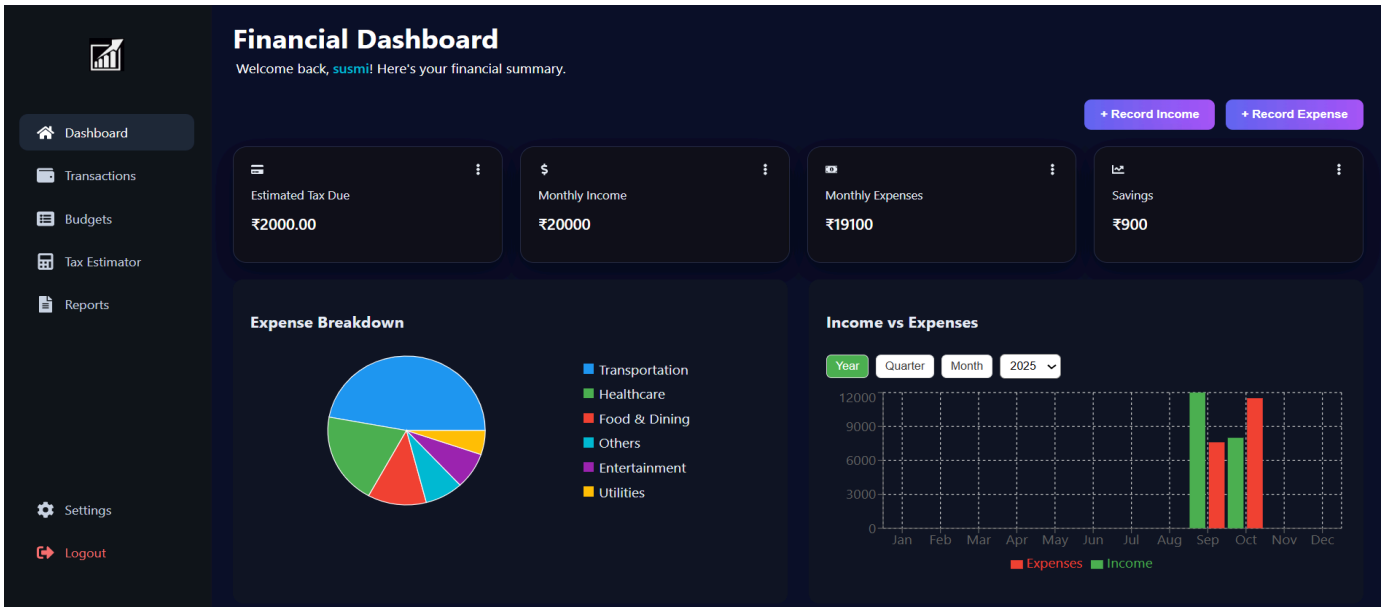
- Date and timestamp of each transaction
- Transaction descriptions
- Categorized spending (Food & Dining, Entertainment, Healthcare, etc.)
- Amounts in Euros with clear Income/Expense classification

Financial Categorization: Automatically organizes transactions into meaningful categories (Food & Dining, Entertainment, Healthcare, Transportation, Utilities) for better spending analysis.

Quick Action Capabilities: Each transaction includes an Actions column enabling users to quickly edit, categorize, or manage individual financial entries.

Real-time Financial Snapshot: Shows current financial flow with both income and expense transactions, helping users maintain awareness of their financial health.

Data Visualization Ready: Structured data format allows for easy integration with charts and graphs in the Budgets and Reports sections for visual financial analysis.



Recent Transactions Component

The Recent Transactions component displays the latest income and expense records of the user, showing key details like category, amount, and date. It helps users quickly track their most recent financial activities in real time.

Key Features:

Centralized Transaction Hub: Offers a single, organized view of all financial activities with the tagline "Manage and review all your financial transactions in one place."

Advanced Filtering & Navigation: Includes tabs (Transactions, Overview) and a "Show Less" toggle, allowing users to customize the level of detail displayed and focus on specific data views.

Full-Action Management: Each transaction row features an "Actions" menu (represented by an icon) , enabling users to edit, categorize, add notes, or delete entries directly from the list.

Integrated Financial Workspace: Seamlessly connects with other modules like Budgets, Tax Estimation, and Reports, accessible via the sidebar, ensuring all financial tools are within reach.

Transaction Component

Description: Allows to view, manage, and review all financial transactions.

Key Features:

Transaction Log: A central table displaying all recorded financial entries

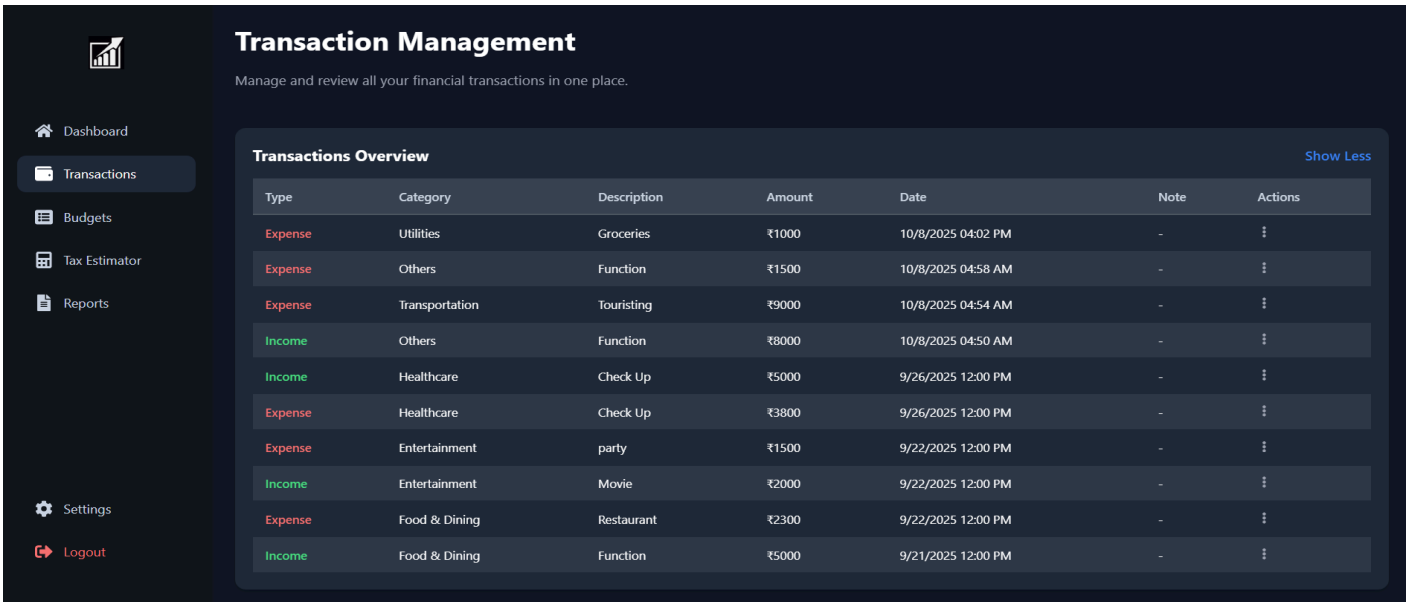
Income/Expense Tracking: Clearly distinguishes between incoming and outgoing funds via a "Type" column (Income/ Expense).

Categorization: Allows transactions to be sorted into categories (e.g., Utilities, Transportation , Healthcare) for better organization and reporting.

Detailed Entries: Each transaction records a description, amount and data.

Data Manipulation: An "Actions" column (implied by the header) suggests users can edit or delete existing transactions.

Comprehensive Data View: Shows a running list of all financial activity, which is essential for bookkeeping and tax preparation.



Budget

This component provides users with a centralized and intuitive interface to create, monitor, and control their budgets accounts.

Key Features:

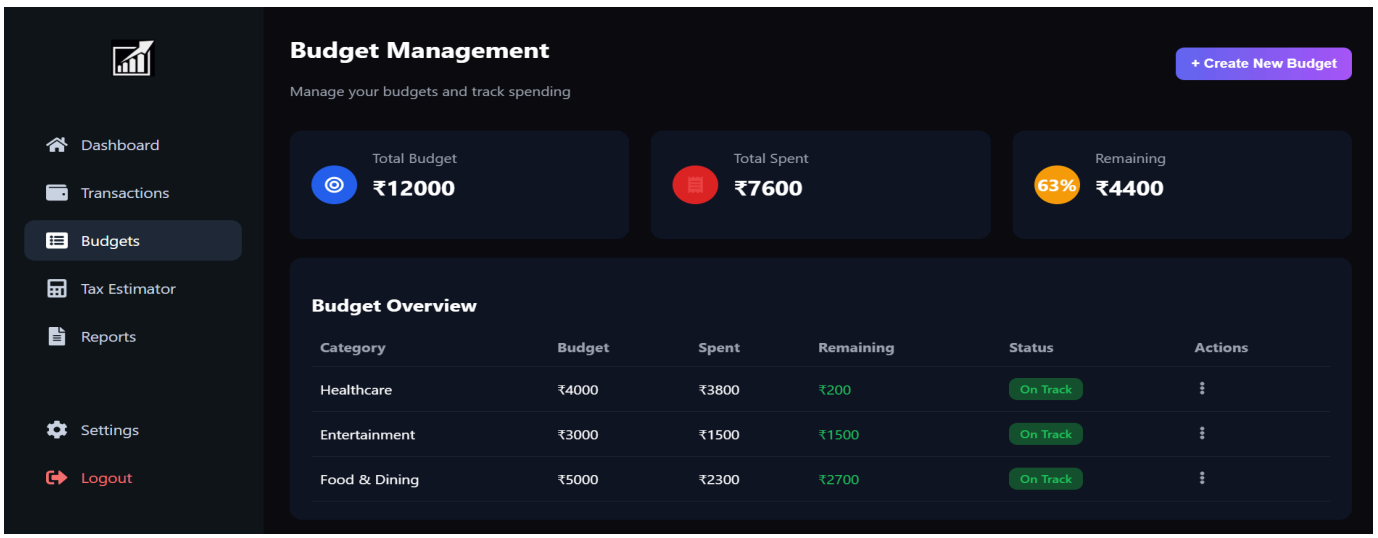
High-Level Financial Summary: Displays key budget metrics at a glance.

Total Budget:The overall allocated budget.

Total Spent:The cumulative amount spent across all categories.

Remaining: The total amount left to spend.

Categorized Budget Overview: A detailed table breaks down the budget by category, showing for each:



Tax Estimator Component

This component provides users with a tool to calculate and forecast their tax obligations based on income, deductions, and other financial data.

Key Features:

Quarterly Tax Calculator: Allows users to estimate taxes for a specific quarter (e.g., Q4), helping with periodic tax planning and payments. Income and Deduction Inputs Includes fields to input financial details such as total income and applicable deductions, which are used to compute the taxable amount.

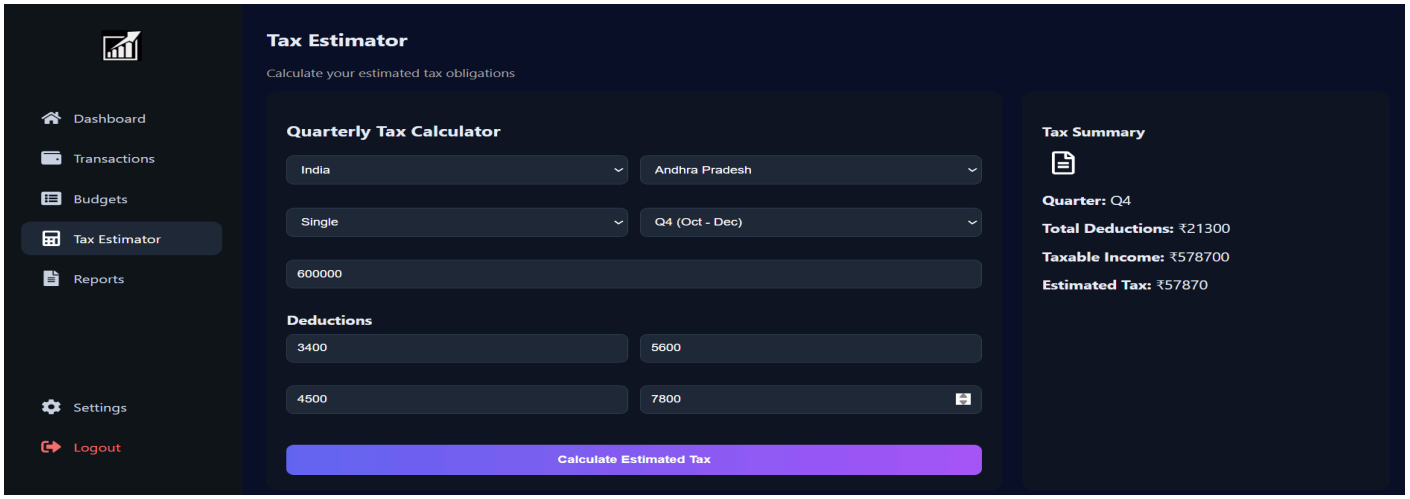
Automated Tax Summary: Generates a clear summary that breaks down:

Total Deductions: The sum of all claimed deductions.

Taxable Income: The income amount subject to tax after deductions.

Estimated Tax: The calculated tax liability based on the taxable income.

Categorized Tax Estimation: Organizes and displays the estimated tax under relevant categories for clarity and reporting. Integrated Financial Overview Easily accessible from the main navigation alongside Transactions, Budgets and Reports, ensuring a seamless workflow for comprehensive financial management.



Tax Calendar Component

To provide an overview and reminders for upcoming tax deadlines.

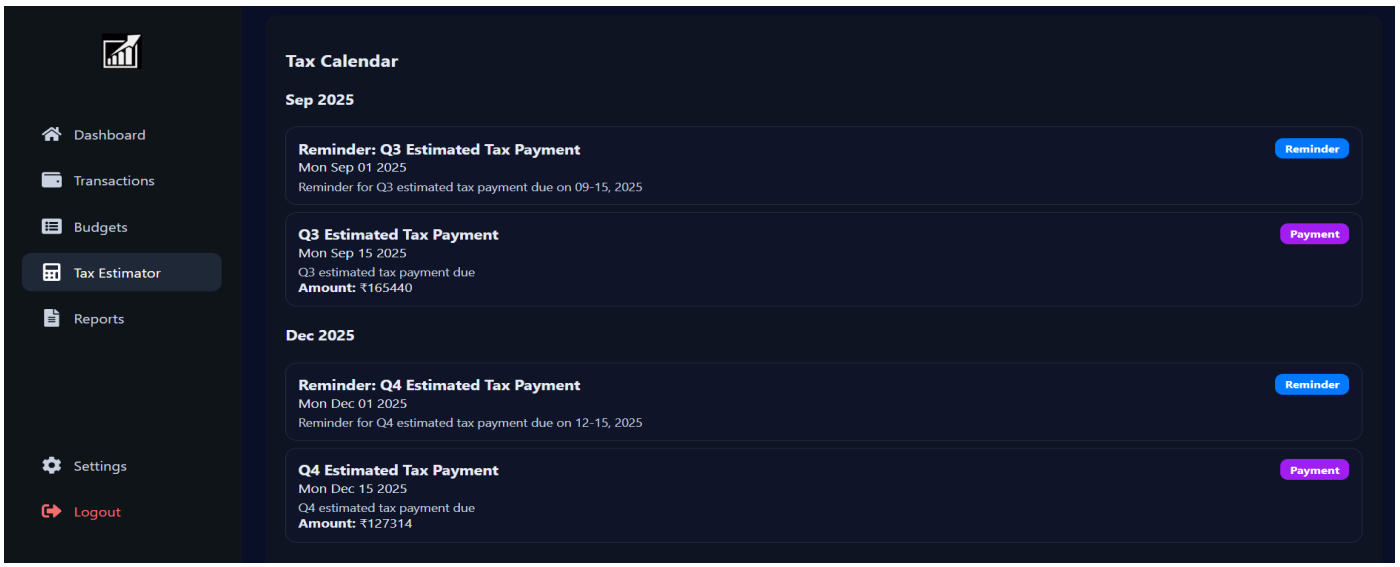
Key Features:

Monthly Calendar View: A high-level view focused on a specific month (e.g., September 2025).

Integrated Dashboard Reminders: Displays upcoming tax-related tasks directly on the main dashboard for easy visibility.

Deadline Tracking: Shows specific deadlines for quarterly estimated tax payments (e.g., "Q3 Estimated Tax Payment Due on Sep 15 2025").

Payment Amount Estimation: Integrates with the Tax Estimator feature to show the calculated amount due for each payment (e.g., "Amount: 1150.440")



Tax History Component

To provide a historical overview of calculated tax estimates and display critical upcoming deadlines.

Key Features:

Prominent Deadline Alert: A highly visible banner at the top of the screen displays the most urgent action item "Estimated tax payment due" with the specific date (Mon Dec 15 2025). The exact Amount due (₹127,314), pulled directly from the latest tax calculation.

Tax Calculation History Table: A central table that logs all previous tax estimator sessions, providing transparency and auditability. Key columns include:

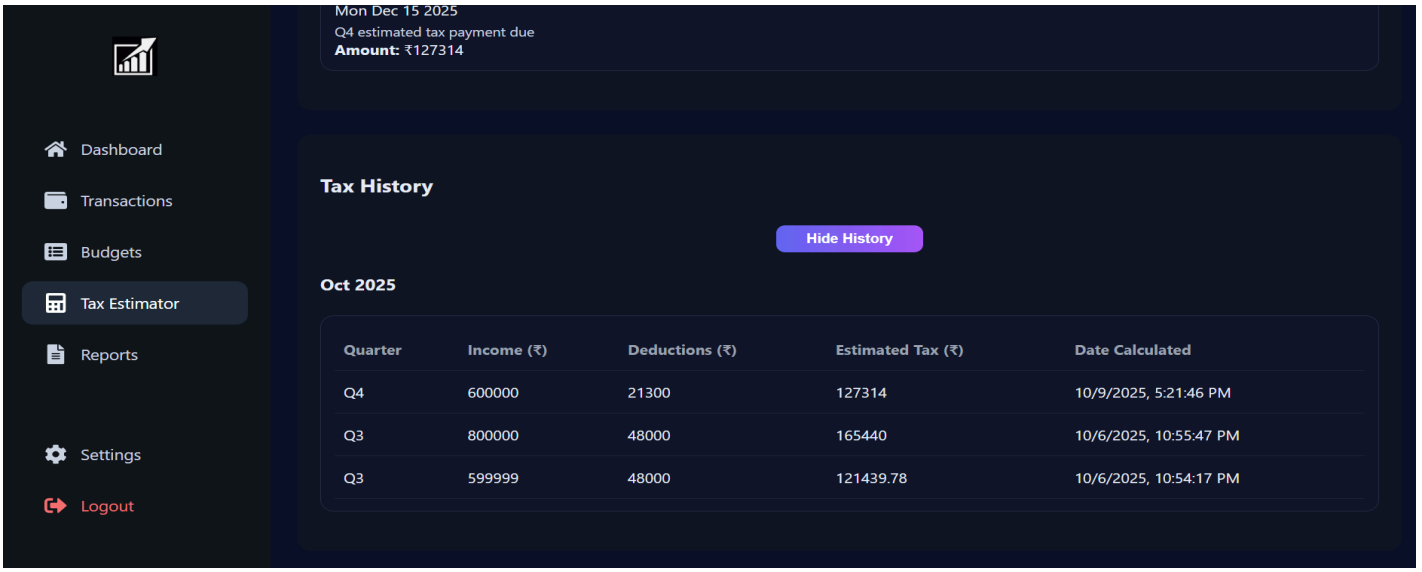
Quarter: The tax period the calculation was for (e.g., Q3, Q4).

Income: The total income entered for the calculation.

Deductions: The total deductions applied.

Estimated Tax: The final calculated tax liability.

Date Calculated: A precise timestamp of when each calculation was performed.



Financial Reports Component

To generate, manage, and access exported financial reports and documents.

Key Features:

Report Repository: A central hub that stores all previously generated financial reports.

Report History Table: A detailed list showing all generated reports with the following information

Report Name: The filename, which often indicates the report type and period a (e.g., tax_list_quarter..., balance sheet...)

Generated Timestamp: The exact date and time the report was created.

Period: Time period the report covers (e.g., list_quarter for a specific quarter, list_month for a specific month).

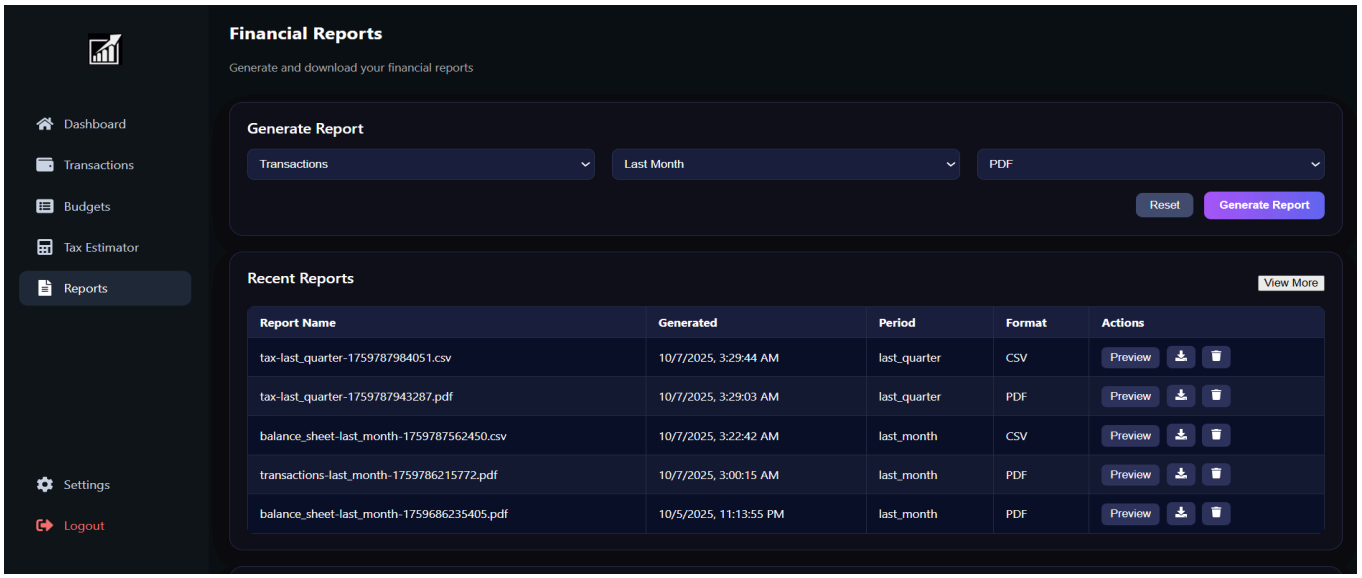
Format: The file format of the exported report, supporting multiple types for flexibility:

- PDF: For a formatted, print-ready document.
- CSV: For raw data that can be opened in spreadsheet applications like Excel.

Report Generation: The presence of a "Generate Report" button (inferred from the context) allows users to create new reports on demand.

Report Actions: An "Actions" column provides buttons (represented by [Picture] icons) to perform tasks like:

- Download: Save the report file to the user's computer.
- View/Preview: Open a quick preview of the report within the application.
- Delete: Remove



Reports & Analytics Component

This component enables users to generate, preview, and download detailed financial reports for analysis and record-keeping.

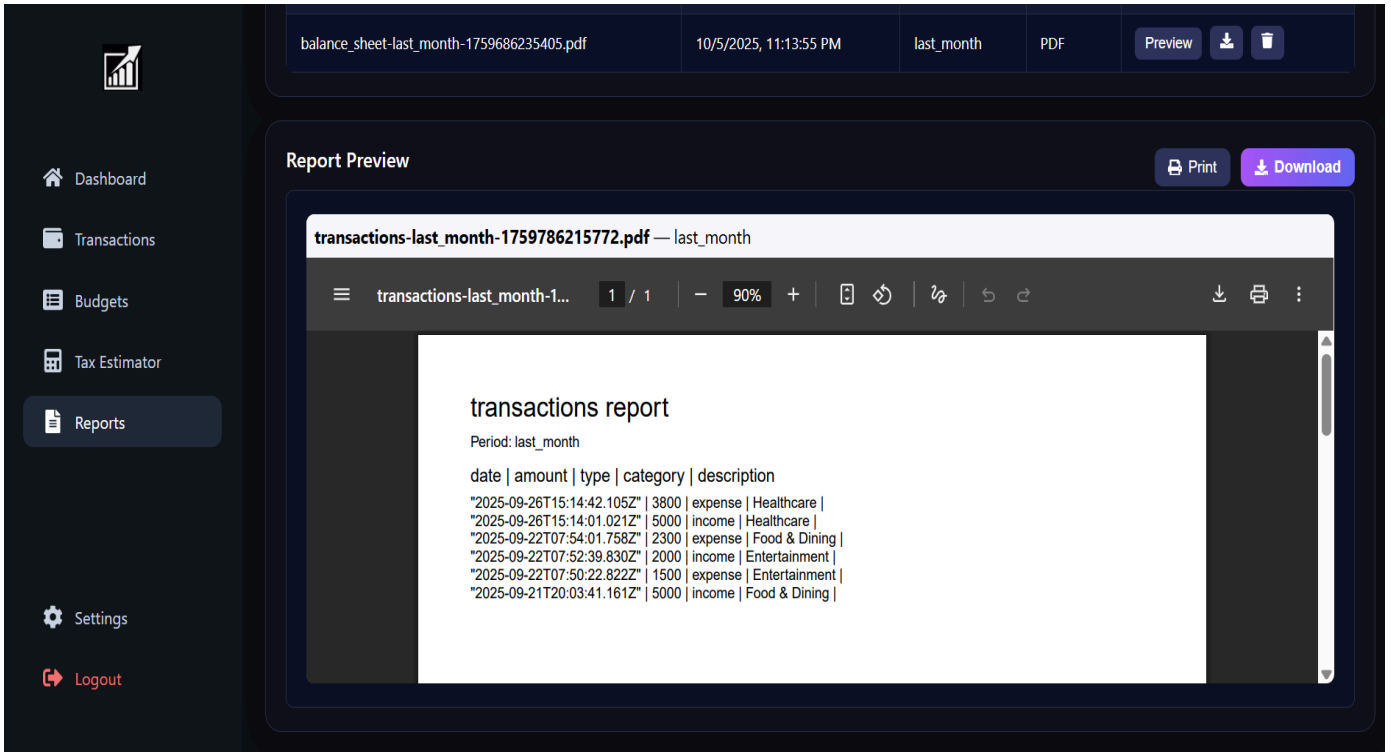
Key Features:

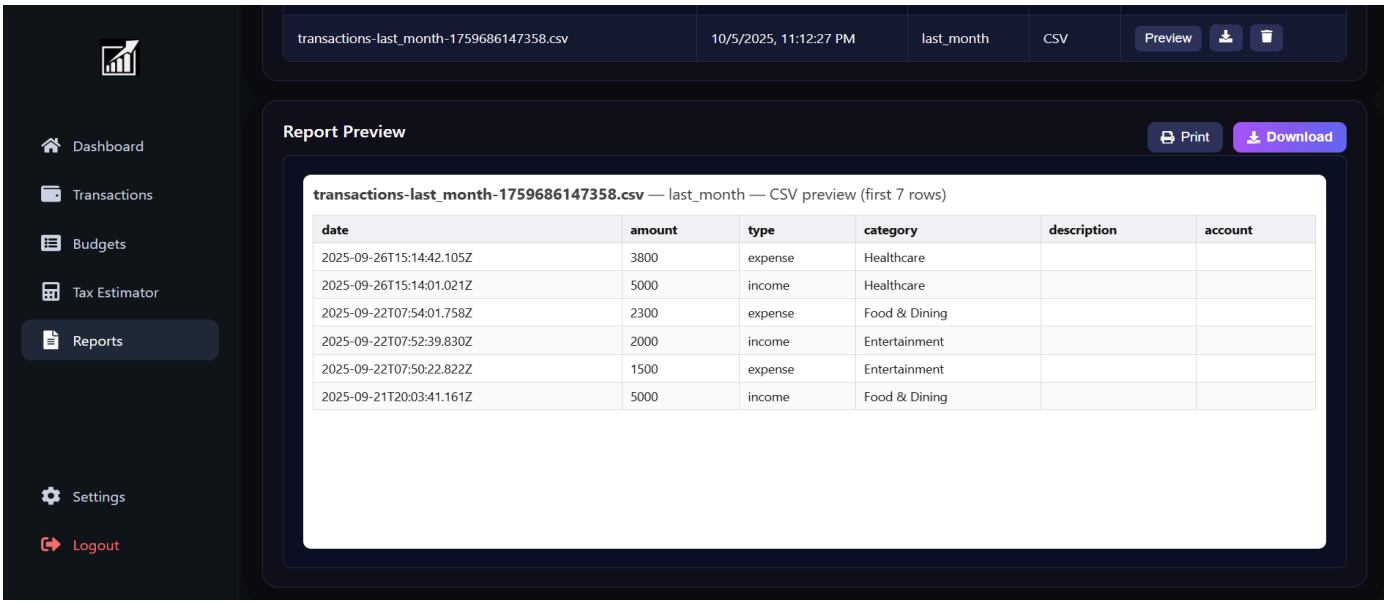
Multi-Format Report Generation: Allows users to generate reports in various formats (PDF, CSV) for different needs (sharing, data analysis).

Historical Report Archive: Maintains a list of recently generated reports with details like name, generation timestamp, period, and format.

Report Preview Functionality: Enables users to preview report contents (both CSV data tables and PDF documents) before downloading.

Period-Based Filtering: Supports generating reports for specific time periods(e.g., last_month, last_quarter).





Settings & Configuration Component

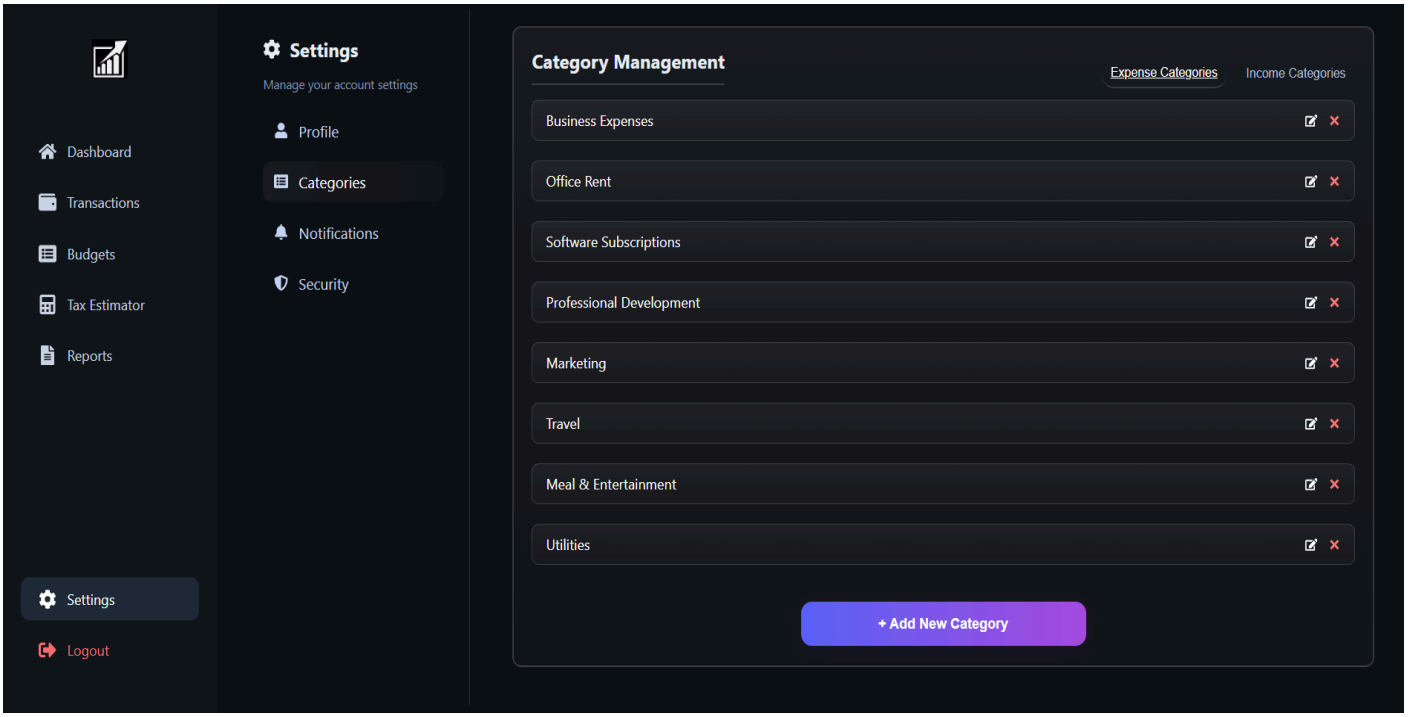
This central hub allows users to customize and manage their account, categories, and application preferences.

Key Features:

Category Management: Provides a dedicated interface to view, add, and manage custom transaction categories (e.g., Office Rent, Software Subscriptions) for personalized budgeting and reporting.

Profile & Security Settings: Offers sections for users to update their profile information and configure security preferences.

Notifications Management: Allows users to control their alert and notification preferences for transactions, budgets and reports.



API Components

User Authentication & Management

User Registration

Route: POST http://localhost:5000/signup
Description: Register a new user account.
Input:

```
{
    "name": "Susmi",
    "email": "susmithak212004@gmail.com",
    "password": "1234",
    "country": "India",
    "income": "Medium (3.5 - 7LPA)"
}
```

Output:
200 OK: { "message": "Signup successful" }
400 Bad Request: { "error": "Invalid input" }

User Login

Route: POST http://localhost:5000/
Description: Authenticate a user.
Input:

```
{
    "email": "susmithak212004@gmail.com",
    "password": "1234"
}
```

Output:
200 OK: { "message": "Login successful"}
401 Unauthorized: { "error": "Invalid credentials" }

Forgot Password Request

Route: POST http://localhost:5000/forgot
Description: Send OTP for password reset.
Input:

```
{
    "email": "susmithak212004@gmail.com"
}
```

Output:
200 OK: { "message": "OTP sent successfully" }
400 Bad Request: { "error": "Failed to send OTP" }

Verify OTP for Password Reset

Route: POST http://localhost:5000/otp
Description: Verify OTP sent to email for password reset.
Input:

```
{
    "otp": "7687"
}
```

Output:
200 OK: { "message": "OTP verified successfully" }
400 Bad Request: { "error": "Invalid OTP" }

Password Reset Request

Route: POST http://localhost:5000/reset
Description: Enter new password for password reset.

Input:

```
{
    "newPassword": "123"
}
```

Output:

200 OK: { "message": "Password Reset successfully" }

400 Bad Request: { "error": "Reset Failed" }

User Logout

Route: POST http://localhost:5000/logout

Description: Logout a user.

Input: None

Output:

200 OK: { "message": "Logged out successfully" }

UI Modules

1. Dashboard

Get Dashboard Summary

Route: GET http://localhost:5000/dashboard

Description: Fetch summary data for charts and analytics.

Input: None (JWT required)

Output:

```
{
   "expenses": [
    { "name": "Investment", "value": 400 },
    { "name": "Food", "value": 300 }
   ],
   " flow": [
   { "m": "01", "v1": 1200, "v2": 900 },
   { "m": "02", "v1": 900, "v2": 700 }
   ]
}
```

2. Transactions

Get All Transactions

Route: GET http://localhost:5000/transactions

Description: Retrieve all transactions for the logged-in user.

Input: None (JWT required)

Output:

```
[
   { "transactionId": "1", "amount": 200, "category": "Food", "type": "expense" },
   { "transactionId": "2", "amount": 500, "category": "Salary", "type": "income" }
]
```

Create Transaction

Route: POST http://localhost:5000/transactions/

Description: Add a new transaction.

Input:

```
{ "amount": 150, "category": "Transport", "type": "expense" }
```

Output:

200 OK: { "message": "Transaction created successfully" }

Update Transaction

Route: PUT http://localhost:5000/transactions/:id

Description: Update transaction by ID.
Input:
{ "amount": 180, "category": "Transport" }
Output:
200 OK: { "message": "Transaction updated successfully" }

Delete Transaction
Route: DELETE http://localhost:5000//transactions/:id
Description: Delete a transaction by ID.
Input: None
Output:
200 OK: { "message": "Transaction deleted successfully" }

3. Budget Management
Get All Budgets
Route: GET http://localhost:5000/budgets/
Description: Retrieve all budgets for the logged-in user.
Input: None (JWT required)
Output:
```
[
  { "budgetId": "1", "name": "Food", "amount": 500 },
  { "budgetId": "2", "name": "Transport", "amount": 300 }
]
```

Create Budget
Route: POST http://localhost:5000/budgets/
Description: Create a new budget entry.
Input:
{ "name": "Entertainment", "amount": 200 }
Output:
200 OK: { "message": "Budget created successfully" }

Update Budget
Route: PUT http://localhost:5000/budgets/:id
Description: Update an existing budget by ID.
Input:
{ "name": "Food & Dining", "amount": 550 }
Output:
200 OK: { "message": "Budget updated successfully" }

Delete Budget
Route: DELETE http://localhost:5000/budgets/:id
Description: Delete a budget by ID.
Input: None
Output:
200 OK: { "message": "Budget deleted successfully" }

4. Tax Estimator
Get Countries
Route: GET http://localhost:5000/tax/countries
Description: Get a list of all countries.
Input: None
Output:
```
[
  { "code": "IN", "name": "India" },
  { "code": "US", "name": "USA" }
]
```

Get States by Country
Route: GET http://localhost:5000/tax/states/:countryCode
Description: Retrieve states for a specific country.
Input: countryCode in URL
Output:

```
[
  { "code": "MH", "name": "Maharashtra" }
]
```

Calculate Tax
Route: POST http://localhost:5000/tax/calculate
Description: Calculate tax based on input data.
Input:

```
{ "income": 50000, "country": "IN", "state": "MH" }
```

Output:

```
{ "taxAmount": 5000 }
```

Get Tax History
Route: GET http://localhost:5000/tax/history
Description: Retrieve user's previous tax calculations.
Input: None
Output:

```
[
  { "date": "2025-01-01", "amount": 5000 }
]
```

Get Tax Calendar
Route: GET http://localhost:5000/tax/calendar
Description: Retrieve tax deadlines and events.
Input: None
Output:

```
[
  { "date": "2025-07-31", "event": "Income Tax Filing Deadline" }
]
```

5. Reports
List Reports
Route: GET http://localhost:5000/reports
Description: Retrieve all available reports.
Input: None
Output:

```
[
{ "reportId": "1", "name": "January Expenses", "type": "PDF" }
]
```

Generate Report
Route: POST http://localhost:5000/reports/generate
Description: Generate a new report.
Input:

```
{ "type": "PDF", "data": { "month": "January" } }
```

Output:

```
200 OK: { "message": "Report generated successfully" }
```

Delete Report
Route: DELETE http://localhost:5000/reports/:id
Description: Delete a report by ID.

Input: None
Output:
200 OK: { "message": "Report deleted successfully" }

Preview Report
Route: GET http://localhost:5000/reports/:id/preview
Description: Preview report in HTML for CSV/PDF.
Input: None
Output: HTML content of the report

6. Health Check
Route: GET /health
Description: Check if the backend server is running.
Output:
{ "ok": true }

7. Settings
Route: GET http://localhost:5000/settings
Description: Manage your account settings.
Output:
```
{
  "message": "Account settings fetched successfully",
    "categories": [
      { "id": 1, "name": "Food" },
      { "id": 2, "name": "Travel" },
      { "id": 3, "name": "Utilities" }
    ]
}
```

Error Handling and Troubleshooting

Effective error handling and troubleshooting ensure the smooth operation of the TaxPal platform. Below are some common errors that users and developers may encounter, along with steps for troubleshooting and resolving them.

NodeMailer Not Working

Error Description: The platform uses NodeMailer to send emails (like OTPs and notifications) through Gmail. If NodeMailer fails, users will not receive emails for verification or alerts.

Troubleshooting Steps:

1. Check Gmail App Password (Most Common Fix)

If your Gmail account has **2-Step Verification enabled**,
➜ Go to your **Google Account** → **Security** → **App Passwords**
➜ Generate an **App Password** (e.g., “for TaxPal”)
➜ Use this **App Password** in place of your regular Gmail password inside your NodeMailer config.

2. Check SMTP Configuration in NodeMailer

Make sure your transporter is correctly set up:
service: `"gmail"`
host: `"smtp.gmail.com"`
port: `587`
secure: `false` (for TLS)

3. Check Environment Variables

Ensure `.env` file contains correct credentials and is loaded properly:

```
EMAIL_USER=youremail@gmail.com

EMAIL_PASS=your_app_password
```

Port Conflicts

Error Description: In the TaxPal backend, the Express server usually runs on port 5000 (or a specified port in the `.env` file).

A **port conflict** occurs when another application (like a previous Node process, MongoDB instance, or React dev server) is already using the same port.

Troubleshooting Steps:

1. Check Which Process Is Using the Port

On Windows:

```
netstat -ano | findstr :5000
```

→ Note the **PID** (Process ID) from the output.

2. Stop the Conflicting Process:

On Windows:

```
taskkill /PID <PID> /F
```

→ This frees up the port so your TaxPal backend can start properly.

3. Change the Port (Alternative Solution)

If you can't stop the conflicting process, modify the backend port in your `.env` or `server.js` file:

.env

```
PORT=5001
```

server.js

```
const PORT = process.env.PORT || 5000;
app.listen(PORT, () => console.log(`Server running on port ${PORT}`));
```

Then restart your server:

```
npm run dev
```

4. Avoid Future Conflicts

Always close running terminals before restarting the backend.
Use a different port for frontend (React default: 3000) and backend (Express: 5000 or 5001).
You can also add `"kill-port"` package for convenience:

```
npx kill-port 5000
```

Database Connection Failures

Error Description: The application may fail to connect to the database due to incorrect configuration or service unavailability.

Troubleshooting Steps:

Check Database Credentials: Ensure that the database username, password, and connection string in the configuration are correct.
For MongoDB, check the connection string format and the database's host URL.
Ensure Database is Running: Make sure that the MongoDB or other database service is running on your local machine or remote server.
Check Database Access Permissions: Ensure that the database user has sufficient permissions to perform operations (e.g., read, write).

API Request Failures

Error Description: API requests may fail due to misconfiguration or incorrect endpoint usage.
Troubleshooting Steps:

Check Endpoint URL: Ensure that the correct API endpoint is being used. Check the route and URL in the backend and the client-side code.

Check API Authentication: If your API requires authentication (e.g., API keys or tokens), ensure that these credentials are correctly included in the request headers.

Examine Response Codes: Check the HTTP status codes returned by the API. Common errors include:

404 (Not Found): The requested endpoint is incorrect or missing.

500 (Internal Server Error): There's an issue with the server; check logs for more details.

401 (Unauthorized): Authentication issues, such as missing or invalid API keys.

Authentication Issues (JWT or Session Timeout)

Error Description: In the TaxPal project, users may face login or session-related issues when their JWT token expires or becomes invalid. This can cause problems such as automatic logout, restricted access to protected pages (like Dashboard, Budgets, or Settings), or API requests failing with a `401 Unauthorized` error.

Troubleshooting Steps:

Check Token Expiry:If you are using JWT tokens, ensure that the token hasn't expired. JWT tokens typically have an expiration time encoded in them.

Verify Token Storage and Retrieval: Ensure the token is correctly stored and attached to all authenticated API requests. If `localStorage` was cleared or token missing, re-login is required.

Ensure Correct Login Credentials: Check that the user is providing the correct username and password and that your backend is validating these correctly.

Check Backend Token Validation: In `authMiddleware.js` (or equivalent), ensure token verification logic is correct. This ensures only valid tokens allow access to protected routes.

Frontend Handling: If token verification fails, automatically redirect the user to the Login page.

Frontend Rendering Issues

Error Description: In the TaxPal project, certain pages (like Dashboard, Budgets, or Settings) may fail to load or display incorrectly if API data is not fetched successfully, React state variables are undefined, or incorrect props are passed to components. Common issues include blank charts, missing recent transactions, or broken layouts caused by missing dependencies or failed API calls.

Troubleshooting Steps:

Check Console for Errors: Use the browser's developer tools (F12) to check for JavaScript errors in the console. Look for error messages related to missing variables, incorrect function calls, or syntax issues.

Verify API Connectivity: Make sure backend APIs (e.g., `/api/budgets`, `/api/transactions`) are running. If not, the frontend may show empty data or fail silently. Check console → Network tab → Response status codes. Ensure correct base URL in `api.js` (`http://localhost:5000` or deployed endpoint).

Ensure All Dependencies Are Installed: to install all required React libraries such as `react-icons`, `recharts`, `axios`, etc.Missing packages can cause rendering failures or import errors.

General Troubleshooting Tips

Clear Cache: Sometimes, browser or application caches can cause issues. Clear the cache or try running the application in incognito mode.

Check Server Logs: Always check the server logs for detailed error messages. They can provide more context on what's going wrong.

Use Debugging Tools: Utilize debugging tools like Chrome DevTools, Visual Studio Code debugger, or backend-specific debuggers to step through the code and identify issues.

Maintenance and Support

Procedures for Regular Maintenance

Backing Up the Database

Use MongoDB's built-in backup tools or cloud-based solutions like MongoDB Atlas to schedule regular backups.

Store backups securely in a cloud storage service (e.g., AWS S3, Google Cloud Storage) to prevent data loss. Rotating API Keys or Credentials

Regularly update sensitive credentials stored in environment variables (e.g., .env files).

Use tools like AWS Secrets Manager or Azure Key Vault for secure storage.

Monitoring Logs and Server Health

Implement logging with tools like Winston or Morgan for application logs.

Use monitoring tools like New Relic, Prometheus, or Datadog to track server performance and uptime.

How Users Can Report Issues or Request Support

Provide a dedicated Support Email Address (e.g., support@taxpalapp.com).

Include a link to a Support Page where users can:
Submit issue reports.
Access self-help guides or FAQs.
Set up a ticketing system using tools like Zendesk or Jira for efficient issue tracking.

Version Control and Updates

Using Git for Version Control

Branching Strategies:

Use the main branch for production-ready code.
Create feature branches for new features or bug fixes.
Use release branches for testing updates before merging into the main branch.

Commit Messages:

Follow a standard format such as [feature]: Add user authentication or [fix]: Resolve login bug.
Applying Updates and Migrations Safely
Test updates in a staging environment before deploying to production.
Perform database migrations using tools like Mongoose migration scripts.

Glossary or FAQ

Glossary

Budget Planner: A tool to track and manage monthly income and expenses.
Transaction History: A log of all recorded incomes and expenses.
Expense Categories: Groups like Food, Utilities, or Entertainment to organize spending.
Savings Tracker: A feature that calculates and monitors your savings over time.
Tax Estimator: A calculator that estimates tax liability based on income and expenses.

FAQs

Why is my dashboard not updating? Ensure you are logged in and refresh the page; API may take a moment to load.
How do I add or edit a transaction? Use the “Add Transaction” button or click the 3-dot menu to edit existing entries.
How can I reset my password? Go to Login, click “Forgot Password,” and follow the email instructions.
Why isn’t the currency showing correctly? Check that your browser and profile settings use INR (₹) as the currency.
How do I logout safely? Click the Logout button in the sidebar to end your session securely.

Account and Profiles

How can I update my profile details?

Log in, go to Settings → Profile, and click “Edit” to update your information.

Why can’t I update my profile?

Ensure all required fields (name, email, country) are filled correctly before saving changes.

Why was my profile information not saved?

Changes may fail if there’s a network issue or session has expired; try logging in again.

Technical Issues

Transactions not updating or showing?

Ensure you are logged in and have a stable internet connection; refresh the dashboard.

I’m not receiving email notifications (e.g., signup, password reset)?

Check the spam/junk folder and verify your email in Settings.

App not loading or showing errors?

Clear browser cache, try a different browser, or ensure the backend server is running.

Cannot delete or edit a transaction?

Make sure your session is active and the transaction exists; refresh if needed.

Select dropdowns or charts not responding?

Check that your browser supports modern JavaScript and the page is fully loaded.

Future Enhancements

Multi-Currency Support: Allow users to manage transactions in different currencies with real-time conversion.
Bank & Payment Gateway Integration: Connect with banks or payment gateways for automatic transaction import.
AI-Powered Expense Insights: Suggest ways to save based on spending patterns using AI algorithms.
Recurring Transaction Management: Automate and track recurring bills, subscriptions, or income.